SDK P2SH QTUM 遗留问题；
修改 BCH 交易报错 BTC 地址的问题，暂未发现，待测试反馈；
Jub_EnumSupportCoins()接口，修改 applet 最小版本号的 bug；
修改qtum的合约地址格式；
量子的 Path 进行特殊处理，无 change 和 address_index：

> We define the following 5 levels in BIP32 path:
>
> ```
> m / purpose' / coin_type' / account' / change / address_index
> ```

SDK 增加 EOS 接口。
- 修改 JUB_GetAddressEOS() applet 地址校验位错误；
- JUB_SignTransactionEOS() 增加 memo 校验显示；
- JUB_SignTransactionEOS() 增加入参 chainID；
- 修改 JUB_SignTransactionEOS() 接口，增加 JUB_BuildActionEOS() 接口，以支持多类型 action（支持 “transfer”、”delegatebw”、“undelegatebw”、“buyram” 和 “sellram” 类型）。

SDK 增加 XRP 接口，仅支持“Direct XRP Payments”类型交易。
SDK BTC 分叉币（BCH/LTC/QTUM）支持 legacy address（即 BTC 地址），涉及到的接口包括：
- JUB_GetAddressBTC
- JUB_SetMyAddressBTC
- JUB_SignTransactionBTC

增加枚举类型 JUB_ENUM_BTC_ADDRESS_FORMAT
BTC applet 对应版本为 1.9.25。新版 SDK 与旧版 BTC applet 兼容，旧版 SDK 与新版 BTC applet 兼容。
==
为了节省空间，增加 **MISC applet，版本 1.0.01**。目前该 applet 支持 EOS 和 XRP 两种数字货币，后续会在此 applet 增加新币种。
由于独立 EOS applet 已发版，SDK 对独立 EOS applet 进行了兼容，但未做独立 XRP 进行兼容，请在使用过程中注意。
兼容策略为：
- 当同时存在独立 EOS applet 和 MISC applet 时，使用 MISC applet 中的 EOS；
- 当仅存在独立 EOS applet 时，使用独立 EOS applet；
- 当仅存在 MISC applet 时，使用 MISC applet，目前支持 EOS 和 XRP；
- SDK 不支持独立 XRP applet。
==
1. 修改 XRP 交易签名无效的问题。同时更新 misc applet，版本 1.0.02。
具体修改内容：
- SDK JUB_GetHDNodeXRP() 和 JUB_GetMainHDNodeXRP() 实现完整，JUB_ENUM_PUB_FORMAT 类型只支持JUB_ENUM_PUB_FORMAT::HEX；ETH 支持 JUB_ENUM_PUB_FORMAT::EXT 和 JUB_ENUM_PUB_FORMAT::XPUB；
- XRP 交易组成中将填充的地址修改为压缩公钥；
2. misc applet 修改：
- amount 和 fee 以 XRP 为单位；
- 增加 00 e6 指令；
- 交易体中检查压缩公钥。
==
1. XRP 新增 memos 的交易体编码，以及 memos 和 destinationTag 在设备上的显示。
- MISC applet v1.0.03；
- 显示方案在 “XRP 接口说明” 部分。

==
1. UniswapV2Router02 合约支持，增加 JUB_SetContrAddrETH 接口。
==
1. 支持 TRX
2. JUB_SetMyAddressXXX 仅支持 BTC 和 ETH，其他币种的该方法均返回 JUBR_IMPL_NOT_SUPPORT
3. 更新 README，framework 打包仍需手工方式，具体方式参考 README 说明
4. 新增 TRX 的 JUB_SetTRC10Asset() 接口，用于设置通证信息，以便 TRC-10 交易显示。

==
1. **新增 FIL。**
2. **支持 ETH 平行链**

本次修改说明：
1. 支持 Uniswap，调整显示需求。

目录：
- ETH JUB_BuildERC20AbiETH 接口说明
- QTUM 接口说明
- EOS 接口说明
- XRP 接口说明
- BTC 分叉币支持 legacy address（即 BTC 地址）
- ETH 支持 UniswapV2Router02 等未知合约方案说明
- TRX 接口说明
- FIL 接口说明
- ETH 平行链接口说明
- 支持 Uniswap

支持 Uniswap 接口说明

显示需求：

合约接口	data 域	屏显	二次确认?
Function: approve(address spender, uint256 amount)	https://hecoinfo.com/tx/0x9475dd4eb6471546962f509ded620fc450392c01821ead48ab06c4ff06beee9e 0x095ea7b3 000000000000000000000000ed7d5f38c79115ca12fe6c0041abb22f0a06c300 ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff	Approve token ANY Token (-> set token) Contract address 0x538cee985e930557d16c383783ca957fa90b63b3(-> to address) ——— 当不确定 Token 信息时，按以下格式显示 Approve token ~~ANY Token (-> set token)~~ Contract address 0x538cee985e930557d16c383783ca957fa90b63b3(-> to address)	是
Function: removeLiquidityWithPermit(address tokenA, address tokenB, uint256 liquidity, uint256 amountAMin, uint256 amountBMin, address to, uint256 deadline, bool approveMax, uint8 v, bytes32 r, bytes32 s)	https://hecoinfo.com/tx/0xeaa375abc391ad1ba5d4f8517bf74d2cbaaad09f99347fdc01528b499970b2b0 0x2195995c 00000000000000000000000058283208294ee02eb147b09117a872616af4f12d 00000000000000000000000089f8e4473842e92441165c3506a0b909049f6faa 00000000000000000000000000000000000000000000000d8d726b7177a7ff06 0000000000000000000000000000000000000000000000862b865ae353cbf655 0000000000000000000000000000000000000000000000015779a9de6eeaffe7 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060b05d67 0000000000000000000000000000000000000000000000000000000000000000 000000000000000000000000000000000000000000000000000000000000001b a157844caca5a15dc11bd8e7584b8532365c3b8beb13c51febf795491a250c69 6ff4b41f7de54c27352e8367ecd65c75c14b723f6bb9b68dde776c601739003b		
Function: swapExactTokensForETH(uint256 amountIn, uint256 amountOutMin, address[] path, address to, uint256 deadline)	https://hecoinfo.com/tx/0x60dd58874480ab3e89e29705a105be029a617287cc83fa4fbf016c24e56759a9 0x18cbafe5 00000000000000000000000000000000000000000000d3c21bcecceda1000000 000000000000000000000000000000000000000000000000000c0696cfff3d3f 00000000000000000000000000000000000000000000000000000000000000a0 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060a8be75 0000000000000000000000000000000000000000000000000000000000000002 00000000000000000000000089f8e4473842e92441165c3506a0b909049f6faa 0000000000000000000000005545153ccfca01fbd7dd11c0b23ba694d9509a6f	Swap ASS for ETH Out 1000000 ASS In ≥ 0.003384944524999999 ETH Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300(Mdex: Router) (ASS contract address) (Wrapped HT(WHT) contract address) ——— 当不确定 Token 信息时，按以下格式显示 ~~Swap ASS for ETH~~ Swap to Token ~~Out 1000000 ASS~~ ~~In ≥ 0.003384944524999999 ETH~~ Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300(Mdex: Router) (ASS contract address) (Wrapped HT(WHT) contract address)	是
Function: swapExactETHForTokens(uint256 amountOutMin, address[] path, address to, uint256 deadline)	https://hecoinfo.com/tx/0x35ea2530f6c6b0fcaa1c811b3e386ba45485aa621f3cabce4e8f7cd88aad7cd2 0x7ff36ab5 000000000000000000000000000000000000000000003b80347549d8da75cc1f 0000000000000000000000000000000000000000000000000000000000000080 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060a75078 0000000000000000000000000000000000000000000000000000000000000002 0000000000000000000000005545153ccfca01fbd7dd11c0b23ba694d9509a6f 00000000000000000000000089f8e4473842e92441165c3506a0b909049f6faa	Swap ETH for ASS Out 0.001(-> value) HT(-> chainID) In ≥ 280984.585739401864530975 ASS Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300(Mdex: Router) (Wrapped HT(WHT) contract address) (ASS contract address) ——— 当不确定 Token 信息时，按以下格式显示 ~~Swap ETH for ASS~~ Swap token ~~Out 0.001(-> value) HT(-> chainID)~~ ~~In ≥ 280984.585739401864530975 ASS~~ Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300(Mdex: Router) (Wrapped HT(WHT) contract address) (ASS contract address)	是
Function: swapExactTokensForTokens(uint256 amountIn, uint256 amountOutMin, address[] path, address to, uint256 deadline)	https://hecoinfo.com/tx/0x968ca5c87759586a68e139729bd1870d636dfd97dbfe9d1b3fa94626b5916858 0x38ed1739 000000000000000000000000000000000000000000000000000000e8d4a51000 000000000000000000000000000000000000000000000000000000000000101a 00000000000000000000000000000000000000000000000000000000000000a0 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060ae02dd 0000000000000000000000000000000000000000000000000000000000000002 00000000000000000000000089f8e4473842e92441165c3506a0b909049f6faa 0000000000000000000000005545153ccfca01fbd7dd11c0b23ba694d9509a6f	Swap ASS for WHT Out 0.000001 ASS In ≥ 0.000000000000004122 WHT Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300(Mdex: Router) (ASS contract address) (Wrapped HT(WHT) contract address) ——— 当不确定 Token 信息时，按以下格式显示 ~~Swap ASS for WHT~~ Swap token ~~Out 0.000001 ASS~~ ~~In ≥ 0.000000000000004122 WHT~~ Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300(Mdex: Router) (ASS contract address) (Wrapped HT(WHT) contract address)	是
Function: withdraw(uint256 wad)	https://hecoinfo.com/tx/0x4304ea8f623a595e97e3d2c0e30f54895c8e21967c0f2f5496385ca65196ff66 0x2e1a7d4d 000000000000000000000000000000000000000000000000011c37937e08000	Withdraw 0.005 HT(-> chainID) Contract address 0x5545153ccfca01fbd7dd11c0b23ba694d9509a6f(Huobi: Wrapped Huobi Token)	是
Function: deposit()	https://hecoinfo.com/tx/0x392efab0fe69e37809b21bbb48658a15d597c1af65415483e2f75565df8072c5 0xd0e30db0	Deposit 0.001(-> value) HT(-> chainID) Contract address 0x5545153ccfca01fbd7dd11c0b23ba694d9509a6f (Huobi: Wrapped Huobi Token)	是
Function: addLiquidityETH(address token, uint256 amountTokenDesired, uint256 amountTokenMin, uint256 amountETHMin, address to, uint256 deadline)	https://hecoinfo.com/tx/0x794d2d9854d7f06b78865d95a9c2fd25c5687bfd2a2ea337de163aaa404ed7fe 0xf305d719 000000000000000000000000b62e3b6a3866f5754fdefcf82e733310e2851043 0000000000000000000000000000000000000000000000000099795532cc01ec 000000000000000000000000000000000000000000000000008273eed193ce6f 000000000000000000000000000000000000000000000000001b4ab8ffe40951 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060ab1e00	(HTMoon (HTMoon) contract address) Add Liquidity 36719216.527920751 HTMoon 0.007681982798760273 ETH(-> chainID) Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300 (Mdex: Router) ——— 当不确定 Token 信息时，按以下格式显示 Add Liquidity ~~36719216.527920751 HTMoon~~ ~~0.007681982798760273 ETH(-> chainID)~~ Contract: 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300 (Mdex: Router)	是
Function: addLiquidity(address tokenA, address tokenB, uint256 amountADesired, uint256 amountBDesired, uint256 amountAMin, uint256 amountBMin, address to, uint256 deadline)	https://hecoinfo.com/tx/0x347dfeb6ba9fb8d9af0c73c2a6ffe9ab19a58fee22b1cf42ee12e6fe4eb9b4bd 0xe8e33700 00000000000000000000000058283208294ee02eb147b09117a872616af4f12d 00000000000000000000000089f8e4473842e92441165c3506a0b909049f6faa 000000000000000000000000000000000000000000084595161401484a000000 00000000000000000000000000000000000000000000152d02c7e14af6800000 00000000000000000000000000000000000000000005ca4ec2a79a7f67000000 00000000000000000000000000000000000000000000000ed2b525841adfc00000 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060b5ab5f	(Medical scientific research (MSR) (ASS contract address) Add Liquidity 7000000 MSR 70000 ASS Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300 (Mdex: Router) ——— 当不确定 Token 信息时，按以下格式显示 Add Liquidity ~~7000000 MSR~~ ~~70000 ASS~~ Contract address 0xed7d5f38c79115ca12fe6c0041abb22f0a06c300 (Mdex: Router)	是
Function: removeLiquidity(address tokenA, address tokenB, uint256 liquidity, uint256 amountAMin, uint256 amountBMin, address to, uint256 deadline)	https://hecoinfo.com/tx/0x7e5051fbfba739ff96ed00d6d17fdb366d6c67bf15a7960012567c3c7469f6a8 0xbaa2abde 00000000000000000000000089f8e4473842e92441165c3506a0b909049f6faa 000000000000000000000000b62e3b6a3866f5754fdefcf82e733310e2851043 0000000000000000000000000000000000000000000000030ff8e0a5810f2d2a 000000000000000000000000000000000000000000005369a0ae2fe741393780 00000000000000000000000000000000000000000000000000185e7834f53aea 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060ab7e92	(ASS contract address) (HTMoon (HTMoon) contract address) Remove Liquidity ≥ 393904.90445028235 ASS ≥ 6859269.818628842 HTMoon ——— 当不确定 Token 信息时，按以下格式显示 Remove Liquidity ~~≥ 393904.90445028235 ASS~~ ~~≥ 6859269.818628842 HTMoon~~	是
Function: removeLiquidityETHSupportingFeeOnTransferTokens(address token, uint256 liquidity, uint256 amountTokenMin, uint256 amountETHMin, address to, uint256 deadline)	https://hecoinfo.com/tx/0xb40a7c46eeec8436d79cec84de29e4a4ae936313d3677928e385fb2e3bfdb06c 0xaf2979eb 000000000000000000000000b62e3b6a3866f5754fdefcf82e733310e2851043 000000000000000000000000000000000000000000000000001a4c8b81466474 000000000000000000000000000000000000000000000000008211048b4d2f21 000000000000000000000000000000000000000000000000001bbb08fffb9d00 0000000000000000000000004f7e578ca0beff5d770160e2b4523ec7d7d48ff3 0000000000000000000000000000000000000000000000000000000060ab2aac	(HTMoon (HTMoon) contract address) Remove Liquidity ≥ 36610458.187018017 HTMoon ——— 当不确定 Token 信息时，按以下格式显示 Remove Liquidity ~~≥ 36610458.187018017 HTMoon~~	是

SDK 接口说明：

增加一新接口 JUB_SetERC20TokensETH()，该接口可一次 `set` 多个 `Token` 信息（至多 2 个），`swap` 或 `addLiquidity` 等的方向性由 `data` 域中 `path` 数组的顺序决定。`Applet` 只需缓存 `Token` 信息并在这些缓存的 `Token` 信息中查找 `contract address` 并做校验。约定当 `set` 之后的签名指令完成，就删除缓存。

```
JUB_RV JUB_SetERC20TokensETH(IN JUB_UINT16 contextID,
                             IN ERC20_TOKEN_INFO tokens[],
                             IN JUB_UINT16 iCount);
```

```
typedef struct stERC20TokenInfo {
    JUB_CHAR_PTR tokenName;
    JUB_UINT16 unitDP;
    JUB_CHAR_PTR contractAddress;
    stERC20TokenInfo() {
        unitDP = 0;
    }
    virtual ~stERC20TokenInfo() {}
} ERC20_TOKEN_INFO;
```

该接口为接口 JUB_SetERC20TokenETH() 的扩展接口，当只需 `set` 一个 `Token` 信息时，也可调用调用 JUB_SetERC20TokenETH() 进行设置。

ETH Uniswap 的签名调用流程（参考 Demo transaction_Uniswap_ETH() 函数流程）：
1. 调用 JUB_CreateContextETH()
2. 调用 JUB_SetERC20TokenETH()/JUB_SetERC20TokensETH() 接口，设置平行链的显示相关信息；
3. 调用 JUB_SignTransactionETH 进行交易。

ETH 平行链接口说明

EIP155
ETH 平行链接口设计原则：与 ETH 兼容，尽量复用 ETH 现有接口，尽量减少开发成本。
平行链对屏显的影响：币种 icon，单位

参照 EIP-20（ERC-20）实现方式，对于平行链显示相关信息，调整现有接口，将 JUB_BuildERC20AbiETH() 拆分成两个接口：
增加一条指令进行设置，保存到 applet 缓存中，待签名结束后，该缓存即无效。

调整前

```
JUB_RV JUB_BuildERC20AbiETH(IN JUB_UINT16 contextID,
                            IN JUB_CHAR_CPTR tokenName,
                            IN JUB_UINT16 unitDP,
                            IN JUB_CHAR_CPTR contractAddress,
                            IN JUB_CHAR_CPTR tokenTo, IN JUB_CHAR_CPTR tokenValue,
                            OUT JUB_CHAR_PTR_PTR abi);
```

调整后

```
JUB_RV JUB_BuildERC20TransferAbiETH(IN JUB_UINT16 contextID,
                                    IN JUB_CHAR_CPTR tokenTo, IN JUB_CHAR_CPTR tokenValue,
                                    OUT JUB_CHAR_PTR_PTR abi);

JUB_RV JUB_SetERC20TokenETH(IN JUB_UINT16 contextID,
                            IN JUB_CHAR_CPTR tokenName,
                            IN JUB_UINT16 unitDP,
                            IN JUB_CHAR_CPTR contractAddress);
```

ETH 平行链的签名调用流程：
1. 调用 JUB_CreateContextETH()
2. 调用 JUB_SetERC20TokenETH() 接口，设置平行链的显示相关信息；
3. 调用 JUB_SignTransactionETH 进行交易。

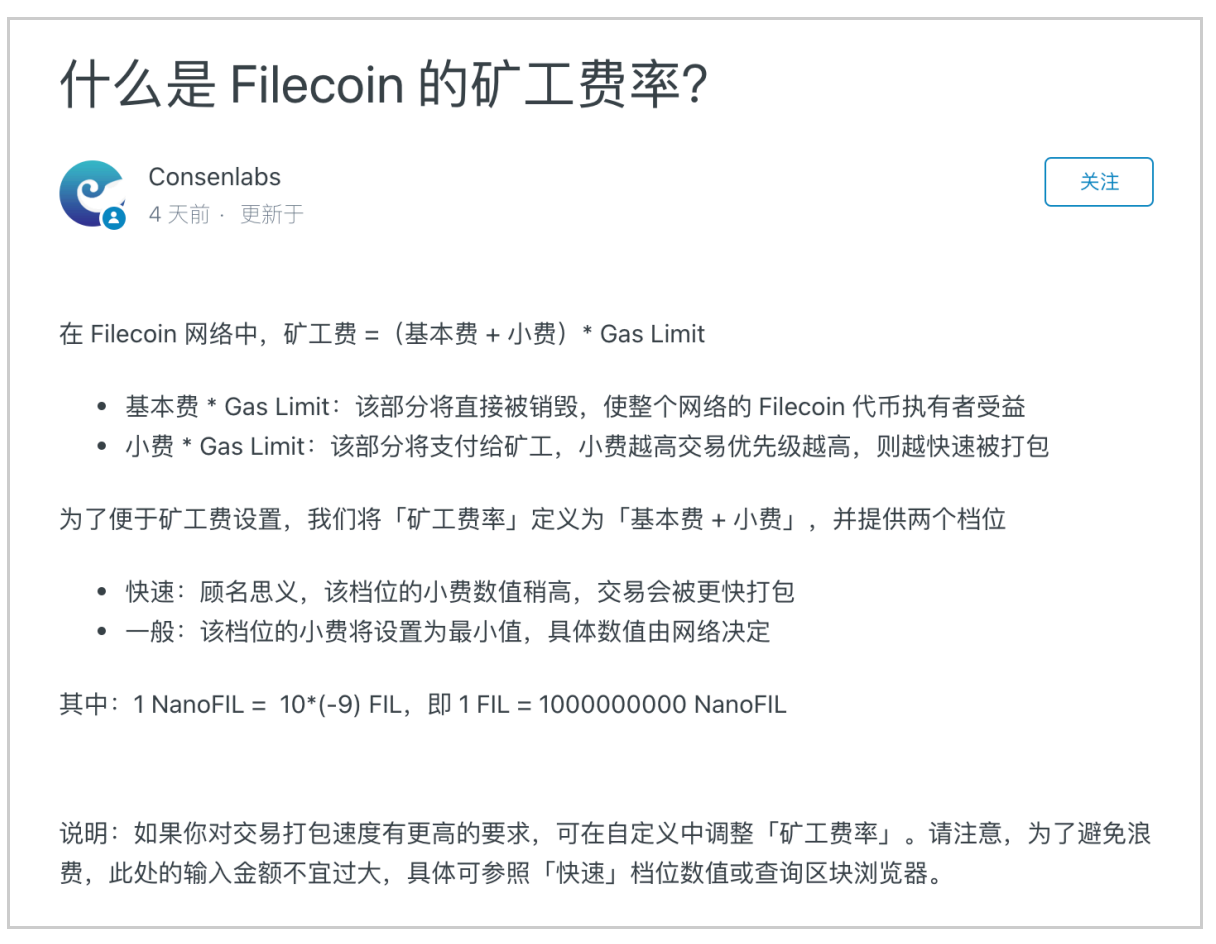
FIL 接口说明

FIL（文件币）的接口类似 ETH，地址格式 SDK 支持转账场景下的解析 f0、f1 和 f3，硬件只支持生成 f1 地址。
- 地址相关接口：

```
JUB_RV JUB_GetAddressFIL(IN JUB_UINT16 contextID,
                         IN BIP44_Path path,
                         IN JUB_ENUM_BOOL bShow,
                         OUT JUB_CHAR_PTR_PTR address);
JUB_RV JUB_GetHDNodeFIL(IN JUB_UINT16 contextID,
                        IN JUB_ENUM_PUB_FORMAT format,
                        IN BIP44_Path path,
                        OUT JUB_CHAR_PTR_PTR pubkey);

JUB_RV JUB_GetMainHDNodeFIL(IN JUB_UINT16 contextID,
                            IN JUB_ENUM_PUB_FORMAT format,
                            OUT JUB_CHAR_PTR_PTR xpub);
```

- 签名接口

```
JUB_RV JUB_SignTransactionFIL(IN JUB_UINT16 contextID,
                              IN BIP44_Path path,
                              IN JUB_UINT64 nonce,
                              IN JUB_UINT64 gasLimit,
                              IN JUB_CHAR_CPTR gasFeeCapInAtto,
                              IN JUB_CHAR_CPTR gasPremiumInAtto,
                              IN JUB_CHAR_CPTR to,
                              IN JUB_CHAR_CPTR valueInAtto,
                              IN JUB_CHAR_CPTR input,
                              OUT JUB_CHAR_PTR_PTR raw);
```



什么是 Filecoin 的矿工费率?

在 Filecoin 网络中，矿工费 =（基本费 + 小费）* Gas Limit

- 基本费 * Gas Limit：该部分将直接被销毁，使整个网络的 Filecoin 代币持有者受益
- 小费 * Gas Limit：该部分将支付给矿工，小费越高交易优先级越高，则越快速被打包

为了便于矿工费设置，我们将「矿工费率」定义为「基本费 + 小费」，并提供两个档位

- 快速：顾名思义，该档位的小费数值稍高，交易会被更快打包
- 一般：该档位的小费将设置为最小值，具体数值由网络决定

其中：1 NanoFIL = 10*(-9) FIL，即 1 FIL = 1000000000 NanoFIL

说明：如果你对交易打包速度有更高的要求，可在自定义中调整「矿工费率」。请注意，为了避免浪费，此处的输入金额不宜过大，具体可参照「快速」档位数值或查询区块浏览器。

TRX 接口说明

```
JUB_RV JUB_SignTransactionTRX(IN JUB_UINT16 contextID,                // contextID
                              IN BIP44_Path path,                     // 签名私钥路径
                              IN JUB_CHAR_CPTR packedContractInPb,    // 待签名数据
                              OUT JUB_CHAR_PTR_PTR rawInJSON);        // JSON 格式的已签名数据
```

TRX 交易流程：
1. 调用 JUB_CreateContextTRX() 获得 TRX 上下文；
2. 准备 TRX 待签名数据：
   - 方法一：可自行准备 protobuf 格式的 message Transaction.raw（Tron.proto）数据；
   - 方法二：调用 JUB_PackContractTRX() 组织待签名数据（SDK 内部组织 message Transaction.raw，但需要外部通过 SDK 自定义结构体传递参数，该接口主要目的是给 JuBiterSDKDemo-iOS 使用）
3. 调用 JUB_SignTransactionXRP() 组织交易，并签名。其中，TxTRX 参数参考 testTRX.json 文件。

注：

关于 AssetContract 类型交易，需在签名之前调用 JUB_SetTRC10Asset() 设置通证信息，以便 applet 注册通证信息。之后遵循步骤2 进行后续 TRX 待签名数据的准备工作。注意，在 JUB_SetTRC10Asset 和 JUB_SignTransactionTRX 之间不要进行其他不必要的操作，以免使通证注册信息无效。

关于 TriggerSmartContract 类型交易，如果 smart contract 为 ERC-20 的 transfer 交易（methodID = a9059cbb），则待签名数据中的 data 域需要调用 JUB_BuildERC20AbiETH，以便向 applet 注册 ERC-20 token 信息。之后遵循步骤2 进行后续 TRX 待签名数据的准备工作。注意，在 JUB_BuildERC20AbiETH 和 JUB_SignTransactionTRX 之间不要进行其他不必要的操作，以免使 ERC-20 注册信息无效。

如果 smart contract 为 TRC-20 类型交易，具体业务不明，则设备屏幕显示 call_value（单位 TRX），合约地址以及 data 域的 hash 前 4 字节，以便用户确认交易信息。

合约 protobuf 定义	packedContractInPb 入参示例	屏显	二次确认?

balance_contract.proto
message TransferContract {
bytes owner_address = 1;
bytes to_address = 2;
int64 amount = 3;
}
	0a0212a222084612a8bc0f1840f340a8fb9cafd22e5a
	65080112610a2d747970652e676f6f676c6561706973
	2e636f6d2f70726f746f636f6c2e5472616e73666572
	436f6e747261637412300a1541e4d98d1c7122a48e4a
	db5954b34af2661f3d52281215417477951026491cfd
	a5920233b818eb3ced770eb4180170cbc799afd22e
		Confirmsending
		1 TRX
		To address:
		TLb2e2uRhzxvrxMcC8VkL2N7zmxYyg3Vfc
		
		（一屏可显示完整）
			是

asset_issue_contract.proto
message TransferAssetContract {
bytes asset_name = 1; // this field is token name before the proposal ALLOW_SAME_TOKEN_NAME is active, otherwise it is token id and token is should be in string format.
bytes owner_address = 2;
bytes to_address = 3;
int64 amount = 4;
}
	0a0212a222084612a8bc0f1840f340a8fb9cafd22e5a
	730802126f0a32747970652e676f6f676c6561706973
	2e636f6d2f70726f746f636f6c2e5472616e73666572
	4173736574436f6e747261637412390a073130303039
	3539121541e4d98d1c7122a48e4adb5954b34af2661f
	3d52281a15417477951026491cfda5920233b818eb3c
	ed770eb4200470cbc799afd22e
		Confirmsending
		4 PEER
		To address:
		TLb2e2uRhzxvrxMcC8VkL2N7zmxYyg3Vfc
		
		fee limit:
		26790816
		
		（需两屏显示）
			是

smart_contract.proto
message TriggerSmartContract {
bytes owner_address = 1;
bytes contract_address = 2;
int64 call_value = 3;
bytes data = 4;
int64 call_token_value = 5;
int64 token_id = 6;
}
	TxID:
	327cb3baddd223f4dce83504ce1cbfc3b9d7b31a1481
	3989604f46530ab58e6e
	
	0a022c8f2208a2ffe52367b1705340c8c084fce82e5a
	ae01081f12a9010a31747970652e676f6f676c656170
	69732e636f6d2f70726f746f636f6c2e547269676765
	72536d617274436f6e747261637412740a15418a4a39
	b0e62a091608e9631ffd19427d2d338dbd121541a614
	f803b6fd780986a42c78ec9c7f77e6ded13c2244a905
	9cbb000000000000000000000000e8137ba81e95b89e
	02c3559a0cba313a4b53632500000000000000000000
	000000000000000000000000000000000000094c5f00
	70a796e0cfe82e900180c2d72f
		以 ERC-20 合约方式显示：
		methodID: a9059cbb
		
		Confirmsending
		156000000 USDT (data.value)
		
		To address:
		TX8K7GXogXRPZnUsxmQrM3ZyHKXQwd93ZQ (data.to)
		
		fee limit:
		100000000
			是

	TxID: b59a8250af45304ab237db0d6d12cd264431be34bb03 f5a32ae218dbba2f42bc 0a02792622089c4c6674fab15e1840a0c592efe82e5a b301081f12ae010a31747970652e676f6f676c656170 69732e636f6d2f70726f746f636f6c2e547269676765 72536d617274436f6e747261637412790a1541d20fc4 52bdde9284ae1656ac6eb5fa372d784a58121541e42d 76d15b7ecd27a92cc9551738c2635c63b71c|1880ade 204|2244a3082be900000000000000000000000000000 00000000000000000000000000000000003200000000 00000000000000000000000000000000000000000000 000000000000170d3fd8eefe82e900180c2d72f	非 ERC-20 合约方式： methodID: a3082be9 Confirmsending 10 TRX（如果 call_value=0，则显示 0） (call_value) To address: TWmhXhXjgXTj87trBufmWFuXtQP8sCWZZV (contract_address) data: xxxxxxxx (data hash) fee limit: 100000000	是

smart_contract.proto
message CreateSmartContract {
bytes owner_address = 1;
SmartContract new_contract = 2;
int64 call_token_value = 3;
int64 token_id = 4;
}
	暂缓实现		

ETH 支持 UniswapV2Router02 等未知合约方案说明

1. SDK 增加接口 JUB_SetContrAddrETH（复用 00 c7 设置 ERC20 token 指令）；

```
- (void)JUB_SetContrAddrETH:(NSUInteger)contextID
                  contrAddr:(NSString*)contractAddress;
```

2. Applet 在 00 c9 ETH 合约交易指令上，对 to 地址进行合约地址检查。如果比对成功，则显示“合约地址”；如果不成功，则 applet 报错不签名；

3. SDK 调用顺序：JUB_SetContrAddrETH -> JUB_SignContractETH。详见 Objective-C demo transaction_UniswapV2Router02_ETH() 接口。

```
{
  "main_path" : "m/44'/60'/0'",
  "chainID" : 1,
  "ETH" : {
    "nonce" : 355,
    "gasLimit" : 167322,
    "gasPriceInWei" : "29000000000",
    "to" : "0x7a250d5630b4cf539739df2c5dacb4c659f2488d",
    "valueInWei" : "0",
    "data" :
"0x18cbafe50000000000000000000000000000000000000000000000004563918244f4000000000000000000000000000000000000000000000000000000030290f654ce64000000000000000000000000000000000000000000000000000000000000000000a0000000000000000000000000d5e27985e3e7c008af184b9b78ddd9b75d32698c000000000000000000000000000000000000000000000000000000005f006cd700000000000000000000000000000000000000000000000000000000000000020000000000000000000000000f5d2fb29fb7d3cfee444a200298f468908cc9420000000000000000000000000c02aaa39b223fe8d0a0e5c4f27ead9083c756cc2",
    "bip32_path" : {
      "change" : false,
      "addressIndex" : 0
    }
  }
}
```

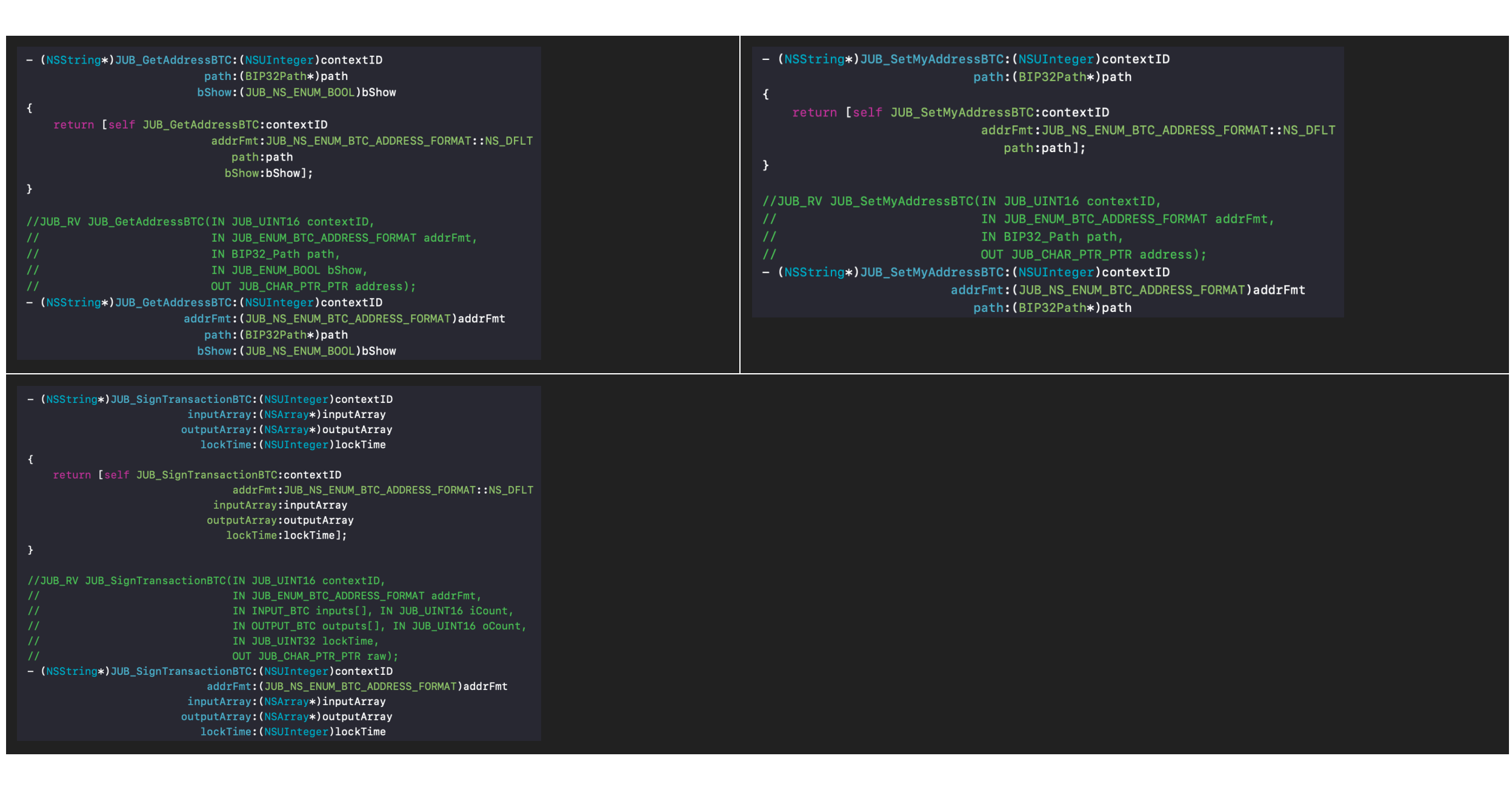
BTC 分叉币支持 legacy address（即 BTC 地址）

BTC applet 对应版本为 1.9.25。新版 SDK 与旧版 BTC applet 兼容，旧版 SDK 与新版 BTC applet 兼容。

C 层定义枚举类型：

```
typedef enum {
    OWN = 0x00,
    LEGACY,
    DFLT = OWN
} JUB_ENUM_BTC_ADDRESS_FORMAT;
```

其中，OWN 为分叉币自己的地址编码格式，LEGACY 为 BTC 地址编码格式。默认使用分叉币自己的地址编码格式。
Object-C 新增三个方法封装，即带 addrFmt 入参的方法。若都使用自己的地址编码格式，则接口不变。若要使用 BTC 地址编码格式，则调用带 addrFmt 参数的方法即可：



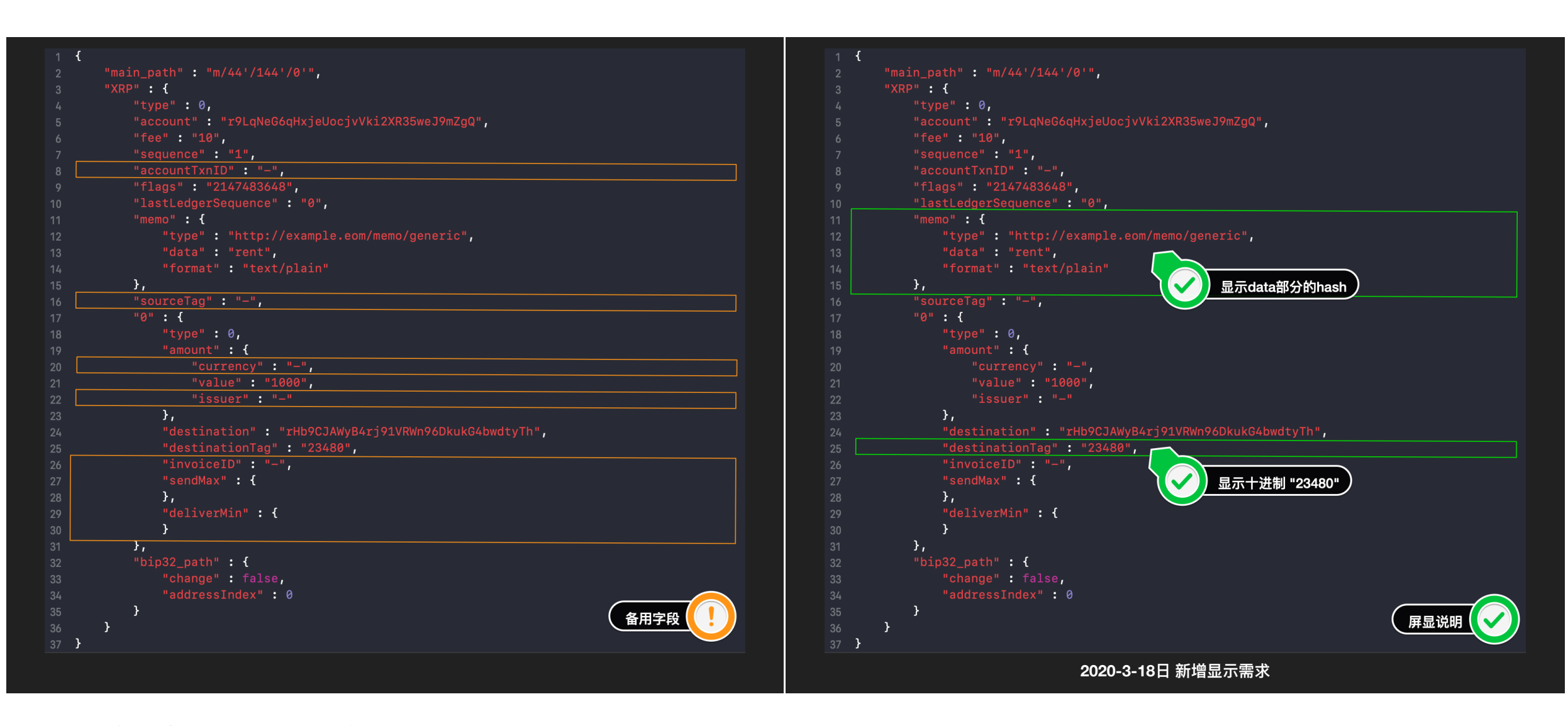
```
- (NSString*)JUB_GetAddressBTC:(NSUInteger)contextID
                          path:(BIP32Path*)path
                         bShow:(JUB_NS_ENUM_BOOL)bShow
{
    return [self JUB_GetAddressBTC:contextID
                           addrFmt:JUB_NS_ENUM_BTC_ADDRESS_FORMAT::NS_DFLT
                              path:path
                             bShow:bShow];
}

//JUB_RV JUB_GetAddressBTC(IN JUB_UINT16 contextID,
//                         IN JUB_ENUM_BTC_ADDRESS_FORMAT addrFmt,
//                         IN BIP32_Path path,
//                         IN JUB_ENUM_BOOL bShow,
//                         OUT JUB_CHAR_PTR_PTR address);
- (NSString*)JUB_GetAddressBTC:(NSUInteger)contextID
                       addrFmt:(JUB_NS_ENUM_BTC_ADDRESS_FORMAT)addrFmt
                          path:(BIP32Path*)path
                         bShow:(JUB_NS_ENUM_BOOL)bShow
```

```
- (NSString*)JUB_SetMyAddressBTC:(NSUInteger)contextID
                            path:(BIP32Path*)path
{
    return [self JUB_SetMyAddressBTC:contextID
                             addrFmt:JUB_NS_ENUM_BTC_ADDRESS_FORMAT::NS_DFLT
                                path:path];
}

//JUB_RV JUB_SetMyAddressBTC(IN JUB_UINT16 contextID,
//                           IN JUB_ENUM_BTC_ADDRESS_FORMAT addrFmt,
//                           IN BIP32_Path path,
//                           OUT JUB_CHAR_PTR_PTR address);
- (NSString*)JUB_SetMyAddressBTC:(NSUInteger)contextID
                         addrFmt:(JUB_NS_ENUM_BTC_ADDRESS_FORMAT)addrFmt
                            path:(BIP32Path*)path
```

```
- (NSString*)JUB_SignTransactionBTC:(NSUInteger)contextID
                         inputArray:(NSArray*)inputArray
                        outputArray:(NSArray*)outputArray
                           lockTime:(NSUInteger)lockTime
{
    return [self JUB_SignTransactionBTC:contextID
                                addrFmt:JUB_NS_ENUM_BTC_ADDRESS_FORMAT::NS_DFLT
                             inputArray:inputArray
                            outputArray:outputArray
                               lockTime:lockTime];
}

//JUB_RV JUB_SignTransactionBTC(IN JUB_UINT16 contextID,
//                              IN JUB_ENUM_BTC_ADDRESS_FORMAT addrFmt,
//                              IN INPUT_BTC inputs[], IN JUB_UINT16 iCount,
//                              IN OUTPUT_BTC outputs[], IN JUB_UINT16 oCount,
//                              IN JUB_UINT32 lockTime,
//                              OUT JUB_CHAR_PTR_PTR raw);
- (NSString*)JUB_SignTransactionBTC:(NSUInteger)contextID
                            addrFmt:(JUB_NS_ENUM_BTC_ADDRESS_FORMAT)addrFmt
                         inputArray:(NSArray*)inputArray
                        outputArray:(NSArray*)outputArray
                           lockTime:(NSUInteger)lockTime
```

XRP 接口使用流程说明

XRP 交易流程：
1. 调用 JUB_CreateContextXRP() 获得 XRP 上下文；
2. 调用 JUB_SignTransactionXRP() 组织交易，并签名。其中，TxXRP 参考 testXRP.json 文件。
JSON 中定义的字段不完全属于 Direct XRP Payments，是备用字段，便于以后扩展支持其他类型交易使用的。



```
{
    "main_path" : "m/44'/144'/0'",
    "XRP" : {
        "type" : 0,
        "account" : "r9LqNeG6qHxjeUocjvVki2XR35weJ9mZgQ",
        "fee" : "10",
        "sequence" : "1",
        "accountTxnID" : "-",
        "flags" : "2147483648",
        "lastLedgerSequence" : "0",
        "memo" : {
            "type" : "http://example.com/memo/generic",
            "data" : "rent",
            "format" : "text/plain"
        },
        "sourceTag" : "-",
        "0" : {
            "type" : 0,
            "amount" : {
                "currency" : "-",
                "value" : "1000",
                "issuer" : "-"
            },
            "destination" : "rHb9CJAWyB4rj91VRWn96DkukG4bwdtyTh",
            "destinationTag" : "23480",
            "invoiceID" : "-",
            "sendMax" : {
            },
            "deliverMin" : {
            }
        },
        "bip32_path" : {
            "change" : false,
            "addressIndex" : 0
        }
    }
}
```

2020-3-18日 新增显示需求

关于 XRP 测试脚本中的 “-”，我想做以下说明：
- 由于调研 XRP 过程中发现 XRP 交易类型非常多，为了便于今后的扩展，在脚本设计以及软件开发过程中预留了许多字段，这些字段大都与当前所支持的 payment 类型交易无关，用“-”做提示，只是提醒开发人员，此项与 payment 类型的交易无关。
- 如果对“-”项进行了设置，SDK 是不关心的，设与不设都没有关系；
- 如果对非“-”项设置为“-”，OC 层的转换会使用“-”的ASCII码作为值进行设置；
- 如果对非“-”项不设置，OC 层会崩溃，因为 OC 层未进行是否为空的检查而直接对其进行转换（该问题后续会加以完善），C层是会对其是否为nullptr进行检查的。
最后强调，“-”只是一种约定俗成的表示方法，程序中并没有对其是否为“-”进行检查。要怎样使得程序健壮，要征求一下大家的意见。

EOS 接口使用说明

EOS 交易流程：
1. 调用 JUB_CreateContextEOS() 获取 EOS 上下文；
2. 调用 JUB_BuildActionEOS() 组织 action，其中，
   - actionArray 为 action 数组，所支持的 action 类型参考枚举类型 JUB_NS_EOS_ACTION_TYPE，定义如下：

```
typedef NS_ENUM(NSInteger, JUB_NS_EOS_ACTION_TYPE) {
    NS_XFER    = 0x00,
    NS_DELE    = 0x01,
    NS_UNDELE = 0x02,
    NS_BUYRAM = 0x03,
    NS_SELLRAM = 0x04,
    NS_ITEM_ACTION_TYPE_NS
};
```

参数设置参考 `testEOS.json`（`transfer`）、`testEOS_stake.json`（资源抵押）、`testEOS_unstake.json`（资源赎回）、`testEOS_buyram.json`（购买ram）以及`testEOS_sellram.json`（卖出ram）等。
3. 调用 JUB_SignTransactionEOS() 进行交易，其中，
   - **chainID**：若为空，则使用现有 EOS chainID(SDK中有)；若不为空，使用传入的 chainID。
   - expiration：超时时间**秒数**
   - referenceBlockId：参考区块ID，即 RPC 的 “head_block_id”
   - referenceBlockTime：参考区块时间戳（**秒数**），即 RPC 的 “head_block_time”（需转换成当前时区时间）
   - actionsInJSON：JSON 格式的 action 数组，由接口JUB_BuildActionEOS()获得。

4. 获取地址调用 `JUB_GetAddressEOS()` 接口

QTUM 接口使用说明

QTUM 交易流程与 BTC 交易流程无异，参考 BTC 交易流程。
QTUM 交易流程参考 Demo -> main_test() case 2301 之 BTC_test()，参数参考 testQTUM.json。

QTUM QRC20 交易流程：
1. 调用 JUB_BuildQRC20Outputs，生成类型为 QRC20 的 output；
2. 准备 input 和 output，注意，其中的 output 仅为一个找零的 output；transaction_qrc20_test()
3. 将类型为 QRC20 的 output 与找零 output 合并为一组 output，作为 JUB_SignTransactionBTC() 入参，进行签名。
QTUM QRC20 交易流程参考 Demo -> main_test() case 2301 之 QTUM_test()，参数参考 testQTUM_qrc20.json。

JUB_BuildERC20AbiETH 接口使用说明

因硬件设备空间有限，原先用 JUB_SetERC20ETHToken 接口增加硬件所不支持的 ERC-20 token。为方便上层业务集成，现将 JUB_SetERC20ETHToken 接口删除，将其功能补充到 JUB_BuildERC20AbiETH 中。

`JUB_BuildERC20AbiETH` 接口定义如下所示：

```
JUB_RV JUB_BuildERC20AbiETH(IN JUB_UINT16 contextID,
                            IN JUB_CHAR_PTR tokenName,
                            IN JUB_UINT16 unitDP,
                            IN JUB_CHAR_PTR contractAddress,
                            IN JUB_CHAR_PTR tokenTo, IN JUB_CHAR_PTR tokenValue,
                            OUT JUB_CHAR_PTR_PTR abi);
```

粗体为新增的三个参数，即 `token` 信息，包括名称、单位(小数点)以及合约地址。

Demo 中，参考调用 transaction_ERC20_ETH() 所示流程。